

# LLM Council (v4.0) Project Report

# Project Report

## Project Name: LLM Council (v4.0 Enterprise Edition)

### 1. Executive Summary

LLM Council successfully bridges the gap between single-model querying and comprehensive multi-model reasoning. The release of version 4.0 has significantly enhanced the system architecture, graduating it from a sophisticated prototype to an enterprise-grade, highly secure FastAPI/Next.js application capable of real-time WebSocket communication, custom agent injection, and robust defense against web vulnerabilities.

### 2. Objectives Achieved

- **Multi-Agent Orchestration**: Successfully implemented a 5-agent debate system using LangGraph. Agents (Sage, Analyst, Strategist, Skeptic, and Judge) effectively pass context, maintain individual personalities, and synthesize final verdicts.
- **Enterprise Security Overhaul**: Secured the application with `slowapi` rate limiting, WebSocket memory exhaust prevention (5MB payload limits), and strict JWT/Pydantic validation schemas.
- **Custom Agent Architecture**: Implemented a fully dynamic Custom Agent pipeline, allowing users to configure bespoke agents with tailored system prompts directly from the UI, persisting them securely in SQLite.
- **Multimodal Context**: Integrated HTML5 canvas and Web Speech API into the Next.js frontend, allowing the backend to receive image attachments, live video frames, and transcribed audio as debate context.
- **Hybrid Execution**: Built the `hybrid_adapter.py` layer, which dynamically routes requests either to OpenRouter's cloud APIs or to local `gguf` weights, providing users with the choice between high-speed cloud execution and absolute local privacy.
- **Real-Time Streaming**: Resolved previous latency issues by utilizing FastAPI WebSockets and LangGraph's streaming capabilities, ensuring the user interface updates instantly as tokens are generated.

### 3. Technical Challenges & Solutions

#### 3.1 Synchronous vs Asynchronous Graph Execution

**Challenge**: LangChain and LangGraph default to synchronous execution which blocks the FastAPI event loop, causing the WebSocket to drop connection or fail to receive intermediate tokens.

**Solution**: Migrated the graph invocation to use `ainvoke` and wrapped the execution in

``asyncio.create_task()``. Implemented custom async stream callbacks to push tokens to the active WebSocket session seamlessly.

## 3.2 Handling Large Multimodal Payloads via WebSockets

**Challenge**: Sending base64 video frames or massive file drops repeatedly via WebSockets caused buffer bloat and server lag, leading to potential DoS vulnerabilities.

**Solution**: Enforced a strict 5MB payload cutoff inside the WebSocket receiver loop. Optimized the frontend to only capture and send keyframes when the user explicitly triggers a "capture" event.

## 3.3 Dynamic Agent Desynchronization

**Challenge**: Creating custom agents with spaces in their names (e.g., "Software Engineer") caused the UI to silently fail during streaming because the backend formatted IDs differently than the frontend.

**Solution**: Discovered during penetration testing and auditing; unified the ID mapping logic across the stack using global string replacement (``replaceAll(' ', '_')``) and robust Pydantic constraints.

## 3.4 Free Tier API Rate Limiting & Failures

**Challenge**: Hitting OpenRouter's free tier endpoints simultaneously for 4 agents resulted in 429 Too Many Requests errors. Furthermore, the primary search engine (DuckDuckGo) frequently blocked automated requests.

**Solution**: Staggered the initial agent analysis using ``asyncio.sleep`` delays. Replaced the asynchronous DuckDuckGo integration with a highly stable thread-safe synchronous wrapper, and built a seamless fallback to the Tavily API if DuckDuckGo fails.

## 4. Future Roadmap

- **Persistent Database Analytics**: Expand the SQLite implementation to store and query historical debate metrics, allowing users to review past verdicts and export them to PDF dynamically.
- **PostgreSQL Migration**: Migrate from SQLite to a dedicated PostgreSQL instance for distributed deployment and massive scalability.
- **Mobile Responsiveness**: Enhance the Tailwind CSS styling to ensure the UI is fully functional and aesthetic on iOS and Android devices.